

TASK 03 — Pagination, Sorting & Filtering

Step 1 — Understand the problem

Imagine your product table contains 1,000,000 products.

If we call:

GET /api/v1/products

the application may try to retrieve all 1,000,000 records.

That is not good.

Suppose the database contains:

Product 1

Product 2

Product 3

...

Product 1,000,000

When we call:

GET /api/v1/products

the application could retrieve a huge amount of data.

Problems

1. Database load

The database has to search/read many records.

2. Memory usage

The application may need to hold many objects in memory.

3. Slow API response

The user has to wait for a large response.

4. Large network response

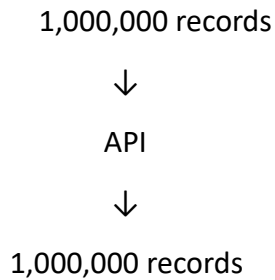
Sending millions of records consumes network bandwidth.

5. Poor user experience

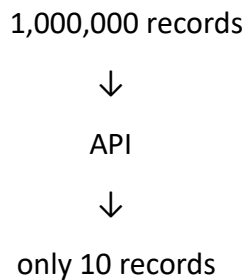
A frontend normally doesn't need 1 million products at once.

Simple idea

Instead of:



we want:



This is called **pagination**.

Step 2 — Learn Pagination

splits large data lists into smaller to boost speed and user experience.

Main types exist:

offset-based (using page numbers or limits/skips)

cursor-based (using a unique ID). [\[1, 2, 3\]](#)

Load More / Infinite Scroll: Replaces numbered pages with a button or scroll event that appends new items dynamically

GET /api/v1/products?page=0&size=10

This is the first pagination API.

Meaning

page = 0

size = 10

means:

Give me the first 10 products.

For example:

Page 0

Product 1

Product 2

...

Product 10

Next:

GET /api/v1/products?page=1&size=10

means:

Page 1

Product 11

Product 12

...

Product 20

Next:

GET /api/v1/products?page=2&size=10

means:

Product 21

...

Product 30

Spring Data pagination uses **zero-based page numbers**.

So:

page=0 → first page

page=1 → second page

page=2 → third page

Step 3 — Configure Spring Data pagination

Pageable

Pageable represents the **pagination request**.

It contains :

- page number

- page size

- sorting

For example:

```
GET /api/v1/products?page=0&size=10
```

Spring converts this into a Pageable object.

Conceptually:

- page = 0

- size = 10

So Pageable tells Spring:

"I want page 0, with 10 records."

Page<T>

Page<T> represents the **paginated result**.

Here: Page<Product>

means:

A page containing Product objects.

For example:

Page<Product> products =

```
productRepository.findAll(pageable);
```

The result contains:

Products

+

Current page information

+

Total records

+

Total pages

Repository Code : Your repository should extend JpaRepository.

```
package com.example.product.repository;

import com.example.product.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ProductRepository
    extends JpaRepository<Product, Long> {
}
```

Because JpaRepository supports pagination, you don't have to manually write pagination SQL for this basic case.

Service Code

Create/use: ProductService.java

```
package com.example.product.service;

import com.example.product.entity.Product;
import com.example.product.repository.ProductRepository;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    public Page<Product> getProducts(Pageable pageable) {
        return productRepository.findAll(pageable);
    }
}
```

```
}  
productRepository.findAll(pageable);  
"Give me only the records requested by this Pageable."
```

Controller Code

```
package com.example.product.controller;  
import com.example.product.entity.Product;  
import com.example.product.service.ProductService;  
import org.springframework.data.domain.Page;  
import org.springframework.data.domain.Pageable;  
import org.springframework.web.bind.annotation.*;  
@RestController  
@RequestMapping("/api/v1/products")  
public class ProductController {  
    private final ProductService productService;  
    public ProductController(ProductService productService) {  
        this.productService = productService;  
    }  
    @GetMapping  
    public Page<Product> getProducts(Pageable pageable) {  
        return productService.getProducts(pageable);  
    }  
}  
public Page<Product> getProducts(Pageable pageable)  
Spring automatically reads: ?page=0&size=10  
and creates the Pageable.
```

Step 4 — Return pagination metadata

```
{  
  "content": [],  
  "page": 0,  
  "size": 10,  
  "totalElements": 100,  
  "totalPages": 10  
}
```

Let's understand each field.

content

```
  "content": []
```

This contains the actual products.

Example:

```
"content": [  
  {  
    "id": 1,  
    "name": "Laptop",  
    "price": 50000  
  },  
  {  
    "id": 2,  
    "name": "Mobile",  
    "price": 20000  
  }  
]
```

page

```
  "page": 0
```

This tells us the current page.

Remember:

0 = first page

1 = second page

2 = third page

size

"size": 10

This means: 10 records were requested per page.

totalElements

"totalElements": 100

This means: There are 100 products in total.

totalPages

"totalPages": 10

Because:

100 products

÷

10 products per page

=

10 pages

So the frontend knows there are 10 pages.

Important Point About Page<Product>

When you directly return:

Page<Product>

Spring Data already provides pagination information.

So this:

@GetMapping

```
public Page<Product> getProducts(Pageable pageable) {  
    return productService.getProducts(pageable);  
}
```

Step 5 — Implement Sorting

Sorting means **arranging products in a particular order**.

GET /api/v1/products?page=0&size=10&sort=price,asc

Parameter	Meaning
page=0	Get the first page
size=10	Get 10 products
sort=price,asc	Sort by price in ascending order

asc mean:

asc = Ascending = Low → High

Example:

15000

25000

50000

desc mean:

desc = Descending = High → Low

Example:

50000

25000

15000

Other sorting examples

price,desc

name,asc

createdAt,desc

Sort by price descending

GET /api/v1/products?sort=price,desc

Most expensive products come first.

Sort by name ascending

GET /api/v1/products?sort=name,asc

Products are arranged alphabetically.

Laptop

Mobile

Tablet

Sort by creation date descending

GET /api/v1/products?sort=createdAt,desc

Recently created products come first.

Step 6 — Implement Filtering

Filtering means **getting only the products that match a condition**.

Filter by Category

GET /api/v1/products?category=mobile

Suppose your database contains:

Name	Category	Price
iPhone	mobile	70000
Samsung	mobile	30000
Dell Laptop	laptop	50000

When you call:

GET /api/v1/products?category=mobile

Response

```
[
  {
    "name": "iPhone",
    "category": "mobile",
    "price": 70000
  },
  {
    "name": "Samsung",
    "category": "mobile",
```

```
"price": 30000
}
]
```

Only mobile products are returned.

Filter by Price Range

GET /api/v1/products?minPrice=1000&maxPrice=10000

Meaning

minPrice = Minimum price

maxPrice = Maximum price

Example database

Name	Price
Mouse	500
Keyboard	2000
Headphones	5000
Monitor	15000

Response

```
[
  {
    "name": "Keyboard",
    "price": 2000
  },
  {
    "name": "Headphones",
    "price": 5000
  }
]
```

Mouse is excluded because it is below 1000.

Monitor is excluded because it is above 10000.

Step 7 — Combine Parameters

GET/api/v1/products?category=mobile&minPrice=100&maxPrice=1000&page=0&size=10&sort=price,asc

Parameter	Meaning	Example
category	Product category	mobile
minPrice	Minimum price	100
maxPrice	Maximum price	1000
page	Page number	0
size	Number of products per page	10
sort	Field and direction	price,asc

Example data

Suppose your database contains:

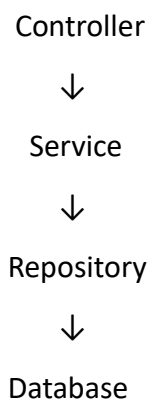
ID	Name	Category	Price
1	iPhone	mobile	900
2	Samsung	mobile	500
3	Laptop	laptop	800
4	OnePlus	mobile	300
5	Tablet	tablet	400

Result

```
{
  "content": [
    {
      "id": 4,
      "name": "OnePlus",
      "category": "mobile",
      "price": 300
    }
  ]
}
```

```
    },  
    {  
      "id": 2,  
      "name": "Samsung",  
      "category": "mobile",  
      "price": 500  
    },  
    {  
      "id": 1,  
      "name": "iPhone",  
      "category": "mobile",  
      "price": 900  
    }  
  ],  
  "page": 0,  
  "size": 10,  
  "totalElements": 3,  
  "totalPages": 1  
}
```

We will use:



Product Entity

```
package com.example.product.entity;

import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;

@Entity
@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String category;
    private BigDecimal price;
}
```

Explanation

`@Entity`

Tells Spring that this class represents a database table.

`@Id`

`@GeneratedValue`

Creates the primary key automatically.

`private BigDecimal price;`

Stores the product price.

Product Repository

```
package com.example.product.repository;

import com.example.product.entity.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.JpaSpecificationExecutor;

public interface ProductRepository
    extends JpaRepository<Product, Long>,
        JpaSpecificationExecutor<Product> {
}
```

JpaSpecificationExecutor:

Because we want to combine conditions such as:

WHERE category = 'mobile'

AND price >= 100

AND price <= 1000

Without Specifications, you would need many separate repository methods.

Product Specification Create:

src/main/java/com/example/product/specification/ProductSpecification.java

Code:

```
package com.example.product.specification;

import com.example.product.entity.Product;
import org.springframework.data.jpa.domain.Specification;
import java.math.BigDecimal;

public class ProductSpecification {

    public static Specification<Product> hasCategory(String category) {
        return (root, query, criteriaBuilder) ->
            category == null || category.isBlank()
                ? null
                : criteriaBuilder.equal(
```

```

        root.get("category"),
        category
    );
}

public static Specification<Product> hasMinPrice(BigDecimal minPrice) {
    return (root, query, criteriaBuilder) ->
        minPrice == null
            ? null
            : criteriaBuilder.greaterThanOrEqualTo(
                root.get("price"),
                minPrice
            );
}

public static Specification<Product> hasMaxPrice(BigDecimal maxPrice) {
    return (root, query, criteriaBuilder) ->
        maxPrice == null
            ? null
            : criteriaBuilder.lessThanOrEqualTo(
                root.get("price"),
                maxPrice
            );
}
}

```

it works:

Suppose the request is:

category = mobile

minPrice = 100

maxPrice = 1000

Spring combines the Specifications:

```
Specification<Product> specification =  
    Specification.where(  
        ProductSpecification.hasCategory(category)  
    )  
    .and(ProductSpecification.hasMinPrice(minPrice))  
    .and(ProductSpecification.hasMaxPrice(maxPrice));
```

This is equivalent to:

```
SELECT *  
FROM products  
WHERE category = 'mobile'  
AND price >= 100  
AND price <= 1000;
```

null means that filter is not applied.

Example:

```
GET /api/v1/products?category=mobile
```

Only the category filter is applied.

Product Service

```
package com.example.product.service;  
  
import com.example.product.entity.Product;  
import com.example.product.repository.ProductRepository;  
import com.example.product.specification.ProductSpecification;  
import lombok.RequiredArgsConstructor;  
import org.springframework.data.domain.*;  
import org.springframework.data.jpa.domain.Specification;  
import org.springframework.stereotype.Service;  
import java.math.BigDecimal;  
  
@Service
```

@RequiredArgsConstructor

```
public class ProductService {  
    private final ProductRepository productRepository;  
    public Page<Product> getProducts(  
        String category,  
        BigDecimal minPrice,  
        BigDecimal maxPrice,  
        int page,  
        int size,  
        String sort  
    ) {  
        Specification<Product> specification =  
            Specification.where(  
                ProductSpecification.hasCategory(category)  
            )  
            .and(ProductSpecification.hasMinPrice(minPrice))  
            .and(ProductSpecification.hasMaxPrice(maxPrice));  
        String[] sortParts = sort.split(",");  
        String sortField = sortParts[0];  
        Sort.Direction direction =  
            sortParts.length > 1 &&  
            sortParts[1].equalsIgnoreCase("desc")  
                ? Sort.Direction.DESC  
                : Sort.Direction.ASC;  
        Pageable pageable = PageRequest.of(  
            page,  
            size,  
            Sort.by(direction, sortField)
```

```

    );
    return productRepository.findAll(specification, pageable);
}
}

```

It happens here:

```
Specification<Product> specification
```

Creates the filters.

```
PageRequest.of(page, size, ...)
```

Creates pagination.

```
Sort.by(direction, sortField)
```

Creates sorting.

```
findAll(specification, pageable)
```

Fetches the filtered, paginated, sorted products.

Product Controller

```

package com.example.product.controller;

import com.example.product.entity.Product;
import com.example.product.service.ProductService;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.Page;
import org.springframework.web.bind.annotation.*;
import java.math.BigDecimal;

@RestController
@RequestMapping("/api/v1/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public Page<Product> getProducts(

```

```

    @RequestParam(required = false)
    String category,
    @RequestParam(required = false)
    BigDecimal minPrice,
    @RequestParam(required = false)
    BigDecimal maxPrice,
    @RequestParam(defaultValue = "0")
    int page,
    @RequestParam(defaultValue = "10")
    int size,
    @RequestParam(defaultValue = "id,asc")
    String sort
) {
    return productService.getProducts(
        category,
        minPrice,
        maxPrice,
        page,
        size,
        sort
    );
}
}

```

Explanation

`@RequestParam(required = false)`

Means the parameter is optional.

`@RequestParam(defaultValue = "0")`

`int page`

If the user does not provide page, Spring uses 0.

```
@RequestParam(defaultValue = "10")  
int size
```

If the user does not provide size, Spring uses 10.

```
@RequestParam(defaultValue = "id,asc")  
String sort
```

If sorting is not provided, products are sorted by ID ascending.

Step 8 — Handle Invalid Parameters

Users may send incorrect values.

Example:

```
GET /api/v1/products?page=-1
```

or:

```
GET /api/v1/products?size=0
```

or:

```
GET /api/v1/products?sort=wrongField,asc
```

Your application should **not accept invalid values blindly**.

Instead, it should return a clear error.

Negative Page

Request

```
GET /api/v1/products?page=-1
```

Problem

Page numbers cannot be negative.

Expected response

```
{  
  "status": 400,  
  "message": "Page number cannot be negative"  
}
```

Zero or Negative Size

Request

GET /api/v1/products?size=0

Problem

Page size must be greater than zero.

Expected response

```
{  
  "status": 400,  
  "message": "Page size must be greater than 0"  
}
```

Invalid Sort Field

Request

GET /api/v1/products?sort=wrongField,asc

Problem

Wrong Field does not exist in the Product entity.

Expected response

```
{  
  "status": 400,  
  "message": "Invalid sort field"  
}
```

Invalid Price Range

Request

GET /api/v1/products?minPrice=1000&maxPrice=100

Problem

Minimum price is greater than maximum price.

Expected response

```
{  
  "status": 400,
```

```
"message": "Minimum price cannot be greater than maximum price"
}
```

Code

We will add validation in the **Service**.

Updated Product Service

```
package com.example.product.service;

import com.example.product.entity.Product;
import com.example.product.repository.ProductRepository;
import com.example.product.specification.ProductSpecification;
import lombok.RequiredArgsConstructor;
import org.springframework.data.domain.*;
import org.springframework.data.jpa.domain.Specification;
import org.springframework.stereotype.Service;
import java.math.BigDecimal;
import java.util.Set;

@Service
@RequiredArgsConstructor
public class ProductService {

    private final ProductRepository productRepository;

    private static final Set<String> ALLOWED_SORT_FIELDS =
        Set.of("id", "name", "category", "price");

    public Page<Product> getProducts(
        String category,
        BigDecimal minPrice,
        BigDecimal maxPrice,
        int page,
        int size,
        String sort
```

```
) {  
    // 1. Validate page  
    if (page < 0) {  
        throw new IllegalArgumentException(  
            "Page number cannot be negative"  
        );  
    }  
    // 2. Validate size  
    if (size <= 0) {  
        throw new IllegalArgumentException(  
            "Page size must be greater than 0"  
        );  
    }  
    // 3. Validate price range  
    if (minPrice != null &&  
        maxPrice != null &&  
        minPrice.compareTo(maxPrice) > 0) {  
        throw new IllegalArgumentException(  
            "Minimum price cannot be greater than maximum price"  
        );  
    }  
    // 4. Validate sort  
    String[] sortParts = sort.split(",");  
    String sortField = sortParts[0];  
    if (!ALLOWED_SORT_FIELDS.contains(sortField)) {  
        throw new IllegalArgumentException(  
            "Invalid sort field: " + sortField  
        );  
    }  
}
```

```

    }
    // 5. Create filters
    Specification<Product> specification =
        Specification.where(
            ProductSpecification.hasCategory(category)
        )
        .and(ProductSpecification.hasMinPrice(minPrice))
        .and(ProductSpecification.hasMaxPrice(maxPrice));
    // 6. Create sorting
    Sort.Direction direction =
        sortParts.length > 1 &&
        sortParts[1].equalsIgnoreCase("desc")
            ? Sort.Direction.DESC
            : Sort.Direction.ASC;
    // 7. Create pagination
    Pageable pageable = PageRequest.of(
        page,
        size,
        Sort.by(direction, sortField)
    );
    // 8. Fetch data
    return productRepository.findAll(specification, pageable);
}
}

```

ALLOWED_SORT_FIELDS?

```

private static final Set<String> ALLOWED_SORT_FIELDS =
    Set.of("id", "name", "category", "price");

```

This means the user can sort only by:

id

name

category

price

Example: GET /api/v1/products?sort=price,asc

Valid.

GET /api/v1/products?sort=name,desc

Valid.

GET /api/v1/products?sort=wrongField,asc

Invalid.

This also prevents unexpected field access.

Step 9 — Test Performance

Now you need to compare:

Fetching all records

GET /api/v1/products

Fetching paginated records

GET /api/v1/products?page=0&size=10

Complete Testing in Postman

Test 1 — Fetch all products

GET http://localhost:8083/api/v1/products

Test 2 — Pagination

GET http://localhost:8083/api/v1/products?page=0&size=10

Expected:

```
{  
  "content": [],  
  "page": 0,  
  "size": 10  
}
```

Test 3 — Filter by category

GET <http://localhost:8083/api/v1/products?category=mobile>

Test 4 — Filter by minimum price

GET <http://localhost:8083/api/v1/products?minPrice=100>

Test 5 — Filter by maximum price

GET <http://localhost:8083/api/v1/products?maxPrice=1000>

Test 6 — Sort by price

GET <http://localhost:8083/api/v1/products?sort=price,asc>

Test 7 — Combine all parameters

GET

<http://localhost:8083/api/v1/products?category=mobile&minPrice=100&maxPrice=1000&page=0&size=10&sort=price,asc>

Test 8 — Invalid page

GET <http://localhost:8083/api/v1/products?page=-1>

Expected:

```
{
  "status": 400,
  "message": "Page number cannot be negative"
}
```

Test 9 — Invalid size

GET <http://localhost:8083/api/v1/products?size=0>

Expected:

```
{
  "status": 400,
  "message": "Page size must be greater than 0"
}
```

Test 10 — Invalid sort

GET <http://localhost:8083/api/v1/products?sort=wrongField,asc>

Expected:

```
{  
  "status": 400,  
  "message": "Invalid sort field: wrongField"  
}
```

Test 11 — Invalid price range

GET <http://localhost:8083/api/v1/products?minPrice=1000&maxPrice=100>

Expected:

```
{  
  "status": 400,  
  "message": "Minimum price cannot be greater than maximum price"  
}
```

Final Deliverables

Pagination

Instead of returning all products at once, return products in **small pages**.

Example

GET [/api/v1/products?page=0&size=10](#)

Give me the first 10 products.

GET [/api/v1/products?page=1&size=10](#)

Give me the next 10 products.

Sorting

Arrange products in **ascending or descending order**.

Example

GET [/api/v1/products?sort=price,asc](#)

Lowest price → Highest price.

GET [/api/v1/products?sort=price,desc](#)

Highest price → Lowest price.

Filtering

Return only products that match a condition.

Example

GET /api/v1/products?category=mobile

Return only mobile products.

GET /api/v1/products?minPrice=100&maxPrice=1000

Return products between ₹100 and ₹1000.

Combined Search API

Combine **filtering + pagination + sorting** in one API.

Example

GET/api/v1/products?category=mobile&minPrice=100&maxPrice=1000&page=0&size=10&sort=price,asc

Validation for Query Parameters

Check whether the user is sending **valid values**.

Examples

Invalid Request	Expected Error
page=-1	Page cannot be negative
size=0	Size must be greater than 0
sort=wrongField,asc	Invalid sort field
minPrice=1000&maxPrice=100	Invalid price range

Example response

```
{  
  "status": 400,  
  "message": "Page number cannot be negative"  
}
```

Postman Collection

A **Postman Collection** is a saved group of API requests.

Instead of creating requests again and again, save them together.

Example collection

Product Service

- |— Get All Products
- |— Get Products - Pagination
- |— Get Products - Sorting
- |— Get Products - Category Filter
- |— Get Products - Price Filter
- |— Combined Search
- |— Invalid Page
- |— Invalid Size
- |— Invalid Sort
- └— Invalid Price Range

Performance Comparison

Compare **fetching all products** with **fetching paginated products**.

Test 1 — Without pagination

GET /api/v1/products

Test 2 — With pagination

GET /api/v1/products?page=0&size=10

We should compare

Comparison	Without Pagination	With Pagination
Records returned	All records	Limited records
Response size	Larger	Smaller
Memory usage	Higher	Lower
Response time	May be slower	Usually faster